МИРЭА
Российский технологический университет

Каталоги и ссылки

Мухортова Н. Н.

Цель

В этом разделе вы научитесь выполнять действия над директориями, а также узнаете, как организованы директории и файлы. В частности, вы научитесь просматривать содержимое директории. Затем вы изучите системные вызовы для добавления и удаления записей о файле. Кроме того, вы научитесь переходить из одной текущей рабочей директории в другую.

Поиск файла

Библиотечная функция pathfind(3G) используется для поиска файла в списке директорий. Аргументы pathfind(3G):

path

Список разделенных двоеточием (:) директорий, в которых ведется поиск. Пустой path трактуется, как текущая директория.

name

Имя файла, который ищется в заданном списке директорий

mode

Задает некоторые характеристики искомого файла. Это строка букв опций из набора rwxfbcdpugks, где буквы задают, соответственно, доступность по чтению, доступность по изменению, доступность по исполнению, нормальный файл, специальный блокориентированный, специальный байт-ориентированный, директорию, FIFO, установку бита идентификатора пользователя, установку бита идентификатора группы, "липкий бит" и ненулевой размер. Опции чтения, изменения и исполнения проверяются для реального идентификатора пользователя и группы.

Если mode - пустая строка, не задается никаких характеристик для поиска файла.

Если файл name со всеми характеристиками, заданными с помощью mode, найден в какойлибо из директорий, перечисленных в path, тогда pathfind(3G) возвращает указатель на строку, содержащую имя директории из path, за которым идет косая черта(/), за которой идет name.

Замечание: Программы, использующие библиотечные функции из секции (3G), должны быть скомпилированы с опцией -lgen, чтобы компилятор подключил библиотеку libgen.a

Поиск файла - Пример

Пример работает следующим образом:

9 Массивы объявляются как static, так чтобы они инициализировались нулями.

17-18 Пользователь пытается ввести mode, которая задает характеристики отыскиваемого файла.

20-21 Пользователь пытается ввести имена директорий для поиска. Точка (.) интерпретируется как текущая директория. Двоеточие (:) также интерпретируется как текущая директория, потому что ее ввод добавляет пустой элемент в список директорий.

23-24 Директории читаются в массив temp.

25-26 Производится проверка, что размер массива, отведенного под хранение списка директорий, не превысил предельного значения.

29 Новая директория вставляется в конец списка директорий поиска.

Поиск файла - Пример

30 В конец списка директорий добавляется двоеточие (:), приготавливая вставку следующей директории.

32 Последнее двоеточие стирается из списка директорий.

33-36 Осуществляется поиск файла, заданного в командной строке, в списке директорий, введенных пользователем. Если файл найден в одной из директорий, то печатается имя этой директории

Программа демонстрируется следующим образом

$ pathfind ls Enter mode (CTRL<D> for no mode): rx

Enter directories to be searched End input with <CTRL/D>

/etc

/bin

/usr/bin

.

/instr/jrs/bin ls is found in /bin/ls

Файл: pathfind.c

```
 5 #define MAXLEN 256
 6
 7 main(int argc, char *argv[])
 8 {
 9    static char temp[50], dir[MAXLEN], mode[10];
10     char *ptr;
...
17     printf("Enter mode (<CRTL/D> for no mode): ");
18     scanf("%s", mode);
19
20     printf("\nEnter directories to be searched\n");
21     printf("End input with <CTRL/D>\n");
22     for(;;) {
23     if (scanf("%s",temp) == -1)
24         break;
25     if (strlen(dir) + strlen(temp) > MAXLEN) {
26         printf("last input not allowed\n");
27         break;
28     }
29     strcat(dir,temp);
30     dir[strlen(dir)] = ':';
31     }
32     dir[strlen(dir) - 1] = '\0';
33     if ((ptr = pathfind(dir,argv[1],mode)) == NULL)
34     printf("\n%s not found\n",argv[1]);
35     else
36     printf("\n%s found in %s\n",argv[1],ptr);
37     exit(0);
38 }
```

Генерация имени для временного файла

Библиотечная функция mktemp(3C) используется для получения уникального имени, которое обычно используется для временных файлов. Аргументом mktemp(3C) является шаблон имени файла, заканчивающийся шестью буквами X, то есть подстрокой XXXXXX. mktemp(3C) заменяет XXXXXX на уникальные символы. При генерации этих символов используются pid процесса и внутренний счетчик для каждого процесса, так что последовательный вызов mktemp(3C) в одном процессе будет порождать разные имена.

Генерация имени для временного файла

mktemp(3C) не проверяет уникальности сгенерированного имени файла. Файл рекомендуется создавать с комбинацией флагов O_CREATE | O_EXCL, и при обнаружении, что такой файл уже существует, повторно вызывать mktemp(3C).

Библиотечные функции tmpfile(3S) и tmpname(3S) также используются для создания имени временного файла

Создание временного файла - Пример

Пример показывает как использовать mktemp(3C) для создания имени временного файла.

9 Объявляется массив tempfile. Он будет содержать имя временного файла.

12-17 Имя временного файла формируется из первых восьми символов имени программы, за которыми следуют подставляемые вместо XXXXXX функцией mktemp(3C) уникальные символы.

19 Печатается имя временного файла.

21-25 Временный файл создается и открывается на чтение и запись.

27 Чтение и запись временного файла опущены.

29 Временный файл уничтожается.

Программа демонстрируется следующим образом:

$ mktemp

temporary file name is: mktempa001SL

$

Файл: mktemp.c

```
 5 #include <fcntl.h>
 6
 7 main(int argc, char *argv[])
 8 {
 9    static char tempfile[15];
10     int fd;
11
12     strncpy(tempfile, argv[0], 8);
13     strcat(tempfile, "XXXXXX");
14     if (mktemp(tempfile) == NULL) {
15     printf("cannot create temporary file name\n");
16     exit(1);
17     }
18
19     printf("temporary file name is: %s\n",tempfile);
20
21     if ((fd = open(tempfile, O_RDWR | O_CREAT,
22     0640)) == -1) {
23     perror(argv[0]);
24     exit(0);
25     }
26
27     /* use temporary file */
28
29     unlink(tempfile);
30     exit(0);
31 }
```

Свойства директории

Директории (каталоги) используются для иерархической организации других файлов, включая и другие директории, а также для организации файловых систем. При монтировании файловой системы необходимо указать существующий каталог, так что после монтирования содержимое файловой системы выглядит как подкаталог или иерархия подкаталогов. Пользовательский процесс может открывать директорию только для чтения. Модификация директории осуществляется неявно, системными вызовами link(2), unlink(2), open(2) и mknod(2).

Формат записи в директории

```
struct dirent {
        ino_t          d_ino;  /* "inode number" of entry */
        off_t          d_off;  /* offset of disk directory entry */
        unsigned short  d_reclen;       /* length of this record */
        char           d_name[1];      /* name of file */
};
```

Первое поле, d_ino, представляет собой номер инода файла. Этот номер определяет инод, который содержит всю управляющую информацию о файле и информацию о его состоянии. Поле d_name[] содержит имя файла. Максимальное количество символов, которое может содержаться в имени файла, зависит от типа используемой файловой системы и может быть определено системным вызовом pathconf(2) с параметром _PC_NAME_MAX.

Поскольку каталог может быть точкой монтирования, максимальную длину имени следует определять для каждого каталога заново.

Обратите внимание, что длина имени файла не учитывается в размере структуры dirent, поэтому при размещении памяти под эту структуру недостаточно выделить sizeof(struct dirent) байт. При размещении памяти для копии уже считанной записи директории можно определять объем требуемой памяти как sizeof(struct dirent)+strlen(d.d_name)

Заметьте, что имя файла не содержится в иноде. Это дает дополнительную гибкость, позволяя иметь несколько записей об одном файле в одной или в различных директориях; имена в различных директориях также могут быть различными. Запись в директории называется жесткой связью.

Директории могут только увеличиваться в размере, так как место, занятое стертой записью, остается в директории. Поэтому могут существовать большие директории с маленьким числом записей. Новые записи используют пространство из-под стертых записей, когда это возможно.

Свойства директории

Права доступа для директорий ведут себя не так, как для файлов:

на чтение: Разрешает считывать содержимое директории. Например, вы можете использовать команду ls(1) без опций для просмотра директории.

на изменение: Позволяет добавлять и удалять файлы в директории. Если на директории установлен «липкий» бит, для удаления файла необходимо быть владельцем файла; иначе, для удаления файла достаточно иметь право записи в каталог

Свойства директории

на исполнение: Позволяет осуществлять поиск в директории, то есть право открывать файлы в этом каталоге и его подкаталогах, а также право проверять существование и доступность таких файлов. Право на исполнение требуется, если вы хотите установить директорию как текущую. Кроме того все компоненты-директории из путевого имени должны быть исполняемыми, хотя и не обязаны иметь право на чтение, когда вы читаете файл или исполняете команду. Требование доступа на исполнение для компонентов директорий путевого имени распространяется на все системные вызовы, которые получают путевое имя в качестве параметра.

Свойства директории

Хотя вы можете не иметь право на чтение из директории, если вы имеете право на исполнение для нее и знаете имена файлов и поддиректорий в этой директории, то вы можете читать или исполнять файлы из нее, в зависимости от прав доступа к файлам.

ls -ld распечатывает информацию о директории (по умолчанию - о текущей директории). Эта команда не выдает информацию о файлах в директории. Такую информацию выдает команда ls -l

Директории и файлы

Следующий пример показывает взаимоотношения между записью в директории, инодом файла и блоками данных. Уникальный номер файла, например, 36 указывает на инод номер 36 в таблице инодов

Директории и файлы

Таблица инодов — это статическая или динамическая таблица на диске, в которой собраны все иноды файловой системы. Формат этой таблицы и формат самого инода зависит от файловой системы; при монтировании файловой системы, ядро инициализирует соответствующий модуль (драйвер файловой системы), который преобразует формат метаданных, хранящихся на диске, во внутренние форматы ядра ОС.

В случае, если файловая система не использует иноды, как FAT 16/32 или CDFS, драйвер файловой системы должен сочинять структуры инода «на ходу»

Директории и файлы

В inode хранятся списки указателей на блоки данных файла. Формат этих списков также зависит от файловой системы. Некоторые ФС хранят отдельный указатель на каждый блок. Для длинных файлов используются косвенные блоки, то есть блоки, хранящие указатели на другие блоки.

Некоторые другие ФС хранят списки так называемых экстентов, то есть фрагментов файла, каждый из которых занимает непрерывное место на диске. Каждый экстент описывается своим логическим смещением от начала файла, указателем на первый блок на диске и длиной.

Директории и файлы

Кроме дисковых файловых систем, Unix поддерживает сетевые файловые системы, а также псевдофайловые системы. Аналогично псевдоустройствам, псевдофайловые системы обслуживаются специальными драйверами, которые во всех наблюдаемых отношениях ведут себя как драйверы обычных ФС, но вместо хранения файлов и каталогов на диске, генерируют содержимое этих файлов и каталогов программно.

Примером псевдофайловой системы является /proc, в которой ядро системы создает по каталогу для каждого процесса в системе. Этот каталог, в свою очередь, содержит образ адресного пространства процесса, ссылки на открытые этим процессом файлы и другую информацию

Изменение текущей директории

Системные вызовы chdir(2) и fchdir(2) изменяют текущую рабочую директорию вашего процесса. Текущая рабочая директория является частью среды исполнения вашего процесса. Текущая директория неявно используется большинством системных вызовов, которые допускают передачу относительного путевого имени: open(2), stat(2), chmod(2) и т. д.

Относительное путевое имя — это имя, которое начинается с любого символа, кроме /. Система ищет файл с таким именем в поддиректориях текущей директории. На самом деле, каждая директория в Unix содержит запись .., которая указывает на родительскую директорию, поэтому относительные имена можно использовать для поиска не только в поддиректориях.

Изменение текущей директории

Параметр chdir(2): path Путевое имя файла-директории, который должен стать новой текущей рабочей директорией.

Параметр fchdir(2): fildes Файловый дескриптор открытого файла. Дескриптор возвращается системным вызовом open(2) и является одним из полей структуры DIR, используемой вызовом opendir(3C).

Изменение текущей директории

Чтобы назначить директорию текущей, процесс должен иметь право записи в эту директорию и все её родительские директории. Оператор shell cd использует chdir(2).

Поскольку текущая директория — это атрибут процесса, команда cd(1) не может быть внешней командой, запускаемой в подпроцессе, как ls(1), mv(1) или cp(1). Это должна быть именно встроенная команда shell, исполняющаяся в том же процессе.

Создание директории

Вызов mkdir(2) создает новую директорию. Параметры mkdir(2):

path Путевое имя файла-директории, которая должна быть создана.

mode Используется для установления прав доступа для директории. Изменяется значением cmask данного процесса

Кроме mkdir(2), каталоги также можно создавать системным вызовом mknod(2).

Команда mkdir(1) вызывает mkdir(2)

Удаление директории

Вызов rmdir(2) удаляет директорию. Параметр rmdir(2) path представляет собой путевое имя файла-директории, которая должна быть удалена. Удаляемая директория не должна быть текущей ни у одного процесса, она не должна содержать записей, кроме . и .., и ни один процесс не должен иметь на эту директорию открытого дескриптора файла. Кроме того, вы должны иметь соответствующие права доступа

Создание и удаление директории - Пример

Этот пример создает директорию и затем удаляет ее. Проверка создания и удаления выполняется вызовом system(3C) для исполнения команды ls(1). Каталог /tmp присутствует во всех Unix-системах и, как ясно из названия, предназначен для хранения промежуточных файлов. При обычных настройках системы, этот каталог доступен для записи всем (чтобы пользователи не могли удалять чужие файлы, обычно на этом каталоге устанавливается «липкий» бит).

Также, обычно, этот каталог очищается при каждой перезагрузке системы. Во многих системах, длительно работающих без перезагрузки, администратор планирует исполняющийся по расписанию скрипт, который удаляет из /tmp файлы, к которым долго не было обращений.

Создание и удаление директории - Пример

В Solaris, каталог /tmp обычно размещается на файловой системе tmpfs, которая размещает файлы не на диске, а в виртуальной памяти. Эта программа работает так:

$ mkrmdir

drwxr-x--- 2 jeg unixc 32 Mar 4 10:50 /tmp/D

/tmp/D not found

Файл: mkrmdir.c

Создание и удаление директории - Пример

```
1 #include <stdlib.h>
2 #include <unistd.h>
3 #include <sys/types.h>
4 #include <sys/stat.h>
5
6 main()
7 {
8     mkdir("/tmp/D", S_IREAD|S_IWRITE|
9         S_IEXEC|S_IRGRP|S_IXGRP);
10
11     system("ls -ld /tmp/D");
12
13     rmdir("/tmp/D");
14
15     system("ls -ld /tmp/D");
16 }
```